MODULE 17 · DOMAIN 2 - CONTAINER ORCHESTRATION

Access control

The 4Cs, how the API server checks every request, RBAC, and ServiceAccounts.

DOMAIN 2 of 4 EXAM WEIGHT 28% MODULE 17 of 35

admin@sthithapragna.com

Every API request answers three questions: **who are you**, **may you do this**, and **is it acceptable**?

The 4Cs of cloud native security

LAYER	PROTECT IT WITH...
Cloud	Provider security, network access to the control plane, node hardening
Cluster	RBAC, admission policy, NetworkPolicy, Secrets encryption, audit logs
Container	Minimal, scanned, signed images; non-root; sandboxed runtimes
Code	Secure coding, dependency scanning, TLS, no secrets in source

ON THE EXAM

Each layer relies on the one outside it. Perfect code cannot save a compromised cloud account.

Currency check: how the docs frame it now

- **The 4Cs** remain the widely taught model, and exam questions use it.
- **Since 2024, kubernetes.io** frames security by lifecycle phase, from the CNCF security white paper.
- **The four phases:** Develop, Distribute, Deploy and Runtime.
- Same ideas, different slicing: layers of the stack, or stages of an application's life.

> ON THE EXAM
>
> Asked for the 4Cs: Cloud, Cluster, Container, Code. Asked for lifecycle phases: Develop, Distribute, Deploy, Runtime.

The path of every API request

01
Request
From kubectl, a pipeline, a controller or a Pod.

›

02
Authentication
Who is this? Certificate, token or OIDC.

›

03
Authorisation
May they do this? Usually RBAC.

›

04
Admission
Mutate, then validate the object.

›

05
Persist
Validated, then stored in etcd.

ON THE EXAM

Fails authentication: 401 Unauthorized. Authenticated but not permitted: 403 Forbidden.

Authentication: who are you?

- **Kubernetes has no user objects.** People are managed outside the cluster.
- **People** authenticate with client certificates, or OIDC tokens from an identity provider.
- **Workloads** authenticate with ServiceAccount tokens.
- **Users belong to groups,** taken from the certificate or the identity provider.
- Unauthenticated requests become the user system:anonymous.

> ON THE EXAM
>
> You cannot kubectl create user. Users come from certificates or an identity provider.

Authorisation modes

MODE	DECIDES BY
`RBAC`	Roles and bindings stored as Kubernetes objects. The standard.
`Node`	Kubelets may only touch objects for Pods on their own node
`Webhook`	Asking an external service
`ABAC`	Attribute rules in a file. Legacy
`AlwaysAllow, AlwaysDeny`	Everything, or nothing. For testing only

ON THE EXAM

kubeadm enables Node and RBAC. Modes are checked in order; if none allows, the request is denied.

RBAC in one slide

- **A subject:** a user, a group, or a ServiceAccount.
- **A rule:** verbs on resources in API groups, such as get, list, watch on pods.
- **A role holds rules; a binding gives a role to subjects.**
- **Permissions only add up.** There are no deny rules; anything not granted is refused.

> ON THE EXAM
>
> RBAC is default-deny and additive. To remove access, remove a binding. You cannot write a deny.

Role versus ClusterRole

ROLE

One namespace

- Lives in a namespace
- Grants access only inside it
- For namespaced resources
- Most application permissions

CLUSTERROLE

Cluster-wide or reusable

- Not in any namespace
- Needed for nodes, PVs, namespaces
- Can be reused in many namespaces
- The built-in view, edit, admin

THE TIEBREAKER

Cluster-scoped resources such as nodes can only be granted through a ClusterRole.

RoleBinding versus ClusterRoleBinding

ROLEBINDING

Grants in one namespace

- Lives in a namespace
- Can reference a Role
- Or a ClusterRole, scoped down to here
- The safe default

CLUSTERROLEBINDING

Grants everywhere

- Cluster-wide
- References a ClusterRole
- Applies in every namespace
- Use sparingly

THE TIEBREAKER

The binding decides the scope. A ClusterRole bound by a RoleBinding only applies in that namespace.

Read-only Pod access for the support team

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: {name: pod-reader, namespace: quayside}
rules:
- apiGroups: [""]
  resources: [pods, pods/log]
  verbs: [get, list, watch]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: {name: support-reads-pods, namespace: quayside}
subjects: [{kind: Group, name: support, apiGroup: rbac.authorization.k8s.io}]
roleRef: {kind: Role, name: pod-reader, apiGroup: rbac.authorization.k8s.io}
```

5-7 Core group, Pods and their logs, read-only verbs.

12-13 The support group gets that Role, in quayside only.

Built-in ClusterRoles

CLUSTERROLE	ALLOWS
`cluster-admin`	Everything, everywhere. Reserve for break-glass use
`admin`	Full control within a namespace, including its Roles and bindings
`edit`	Read and write most objects in a namespace; not Roles or bindings
`view`	Read-only on most objects; not Secrets

> ON THE EXAM
>
> Bind view, edit or admin with a RoleBinding to give one namespace's worth of access.

People have accounts outside the cluster.
Workloads have ServiceAccounts inside it.

ServiceAccounts

- **A namespaced object.** Every namespace starts with one called default.
- **Each Pod runs as one,** set by serviceAccountName, or default if unset.
- **Bound to roles like any subject.** Its username: system:serviceaccount:quayside:booking-api.
- **Default has no special permissions,** and should stay that way.

> ON THE EXAM
>
> A Pod's permissions are exactly those bound to its ServiceAccount.

Legacy versus projected ServiceAccount tokens

LEGACY SECRET TOKENS

Long-lived

- Stored in a Secret
- Never expired
- No longer created automatically since 1.24
- A stolen one worked forever

PROJECTED TOKENS

Short-lived and bound

- Issued by the TokenRequest API
- Typically valid for about an hour
- Rotated automatically by the kubelet
- Invalid once the Pod is gone

THE TIEBREAKER

Modern clusters mount short-lived, audience-bound tokens. Old material describing Secret tokens is dated.

Hardening ServiceAccounts

- **One ServiceAccount per workload,** with only the permissions it needs.
- **automountServiceAccountToken: false** for Pods that never call the API.
- **Never bind broad roles** such as cluster-admin to a default ServiceAccount.
- **Check with kubectl auth can-i,** impersonating the ServiceAccount.

> ON THE EXAM
>
> A compromised Pod can do whatever its ServiceAccount can do. Keep that small.

Checking permissions with auth can-i

```
1 $ kubectl auth can-i create deployments -n quayside
2 yes
3
4 $ kubectl auth can-i list secrets -n quayside \
5     --as=system:serviceaccount:quayside:booking-api
6 no
7
8 $ kubectl auth can-i --list -n quayside
```

1 Can I do this?

4-5 Can this ServiceAccount do it? Impersonation, with --as.

8 Everything I may do in this namespace.

> Quayside's support team needs to read Pods and their logs in the quayside namespace. They must not change anything.
>
> They must not see anything in any other namespace.

Which RBAC setup fits?

A The view ClusterRole with a ClusterRoleBinding

B A read-only Role and a RoleBinding in quayside

C cluster-admin, removed again after a month

D The edit ClusterRole bound by a RoleBinding

◆ PAUSE AND DECIDE

Which RBAC setup fits?

A The view ClusterRole with a ClusterRoleBinding
Read-only, but in every namespace.

B A read-only Role and a RoleBinding in quayside
Only the verbs needed, only in quayside.

C cluster-admin, removed again after a month
Everything, everywhere, for a month.

D The edit ClusterRole bound by a RoleBinding
Right namespace, but allows changes.

THE SIGNAL

One namespace, read-only: Role plus RoleBinding in that namespace.

> The same read-only Pod access is now needed in twelve of Quayside's twenty namespaces.
>
> The platform team wants one definition of the permissions, not twelve copies.

How should they set it up?

A One ClusterRole with one ClusterRoleBinding

B A copy of the Role and a RoleBinding in each namespace

C A ClusterRoleBinding plus deny rules for eight namespaces

D One ClusterRole, a RoleBinding in each of the twelve

◆ PAUSE AND DECIDE

How should they set it up?

A One ClusterRole with one ClusterRoleBinding

One definition, but it applies in all twenty namespaces.

B A copy of the Role and a RoleBinding in each namespace

Works, but twelve copies is what they want to avoid.

C A ClusterRoleBinding plus deny rules for eight namespaces

RBAC has no deny rules.

D One ClusterRole, a RoleBinding in each of the twelve

Defined once; each RoleBinding scopes it to one namespace.

THE SIGNAL

Reuse without copying: a ClusterRole, bound per namespace with RoleBindings.

> An attacker compromised a booking API Pod and used its token to list every Secret in the cluster.
>
> The investigation finds the Pod ran as the default ServiceAccount, which someone had bound to cluster-admin.

What change would most directly have prevented this?

A Rotating ServiceAccount tokens far more frequently

B A NetworkPolicy limiting the Pod's traffic

C A dedicated ServiceAccount with minimal permissions

D Encrypting Secrets at rest in etcd

◆ PAUSE AND DECIDE

What change would most directly have prevented this?

A Rotating ServiceAccount tokens far more frequently

A fresh token with cluster-admin is just as dangerous.

B A NetworkPolicy limiting the Pod's traffic

Useful, but the Pod still needs to reach the API server.

C A dedicated ServiceAccount with minimal permissions

The token could only do what the booking API needs.

D Encrypting Secrets at rest in etcd

The API server still returns them decrypted to anyone allowed.

THE SIGNAL

The blast radius of a compromised Pod is its ServiceAccount's permissions.

> A developer logs in through Quayside's identity provider and runs kubectl get pods -n staging.
>
> The API server returns 403 Forbidden. kubectl auth can-i list pods -n staging says no.

Which stage rejected the request?

A Authorisation: known user, no permission

B Authentication: the user was not recognised

C Admission control: the object failed validation

D etcd: the Pods could not be read from storage

◆ PAUSE AND DECIDE

Which stage rejected the request?

A Authorisation: known user, no permission

403 means authenticated but not allowed: RBAC said no.

B Authentication: the user was not recognised

That returns 401 Unauthorized, not 403.

C Admission control: the object failed validation

Admission checks writes. A get creates no object.

D etcd: the Pods could not be read from storage

Storage errors are not 403s, and can-i agrees it is a permission issue.

THE SIGNAL

401: who are you? 403: I know you, and the answer is no.

Module 17 in five lines

- **4Cs:** Cloud, Cluster, Container, Code. Docs now also use Develop, Distribute, Deploy, Runtime.
- Requests pass **authentication, authorisation, admission**. 401 versus 403.
- **RBAC is additive, default-deny:** Roles and ClusterRoles, bound by RoleBindings and ClusterRoleBindings.
- **The binding decides the scope;** a ClusterRole via RoleBinding is one namespace only.
- **ServiceAccounts** for workloads, with short-lived projected tokens. One per workload.

RBAC decides who may create a Pod. Next: **what that Pod may do** once it runs.